

How Does Python Work?

Tim Lin

y10124@princeton.edu

July 13, 2026



Initial Framing

Python is powerful... and fast; plays well with others; runs everywhere; is friendly and easy to learn; is Open. – Python.org

In Python, you usually don't manage memory, registers, stack layout, or other low-level details. You manage objects, names, functions, modules, and interfaces. – ChatGPT, 2026

Anatomy of Computation

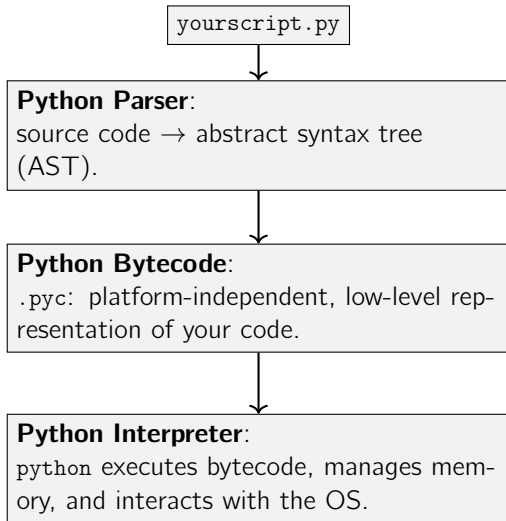
Python is:

- ▶ *High-level*
- ▶ *Interpreted*

Two-stage computation: Python source code is *compiled* into bytecode, which is then *executed* by the **Python interpreter**.

This is to be contrasted with languages like C/C++.

source code $\rightarrow$ machine code $\rightarrow$ CPU execution.



Python's Runtime Model: Names and Objects

Python is an object-based language.

Objects are abstractions that contain data and support operations.

Variables are names bound to objects.

Every object has:

- ▶ **identity**: which object is this?
- ▶ **type**: what operations does it support?
- ▶ **value**: what data/state does it currently contain?

Example

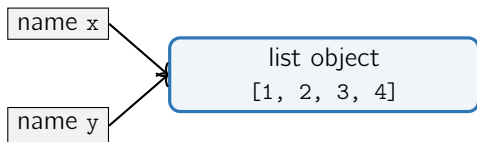
```
x = [1, 2, 3]
```

```
y = x
```

```
y.append(4)
```

```
print(x)    ⇒    [1, 2, 3, 4]
```

Why? Both names point to the same list object.



Python's Runtime Model: Names and Objects

Python is an object-based language.

Objects are abstractions that contain data and support operations.

Variables are names bound to objects.

Every object has:

- ▶ **identity**: which object is this?
- ▶ **type**: what operations does it support?
- ▶ **value**: what data/state does it currently contain?

Example

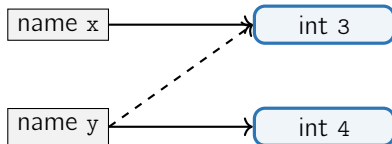
```
x = 3
```

```
y = x
```

```
y = y + 1
```

```
print((x, y))    ⇒    3, 4
```

Why? You re-bound `y` to a new integer object.



Assignment **binds names**. Mutation **changes objects**.

Outcome: Dynamic Typing

Names are flexible

```
x = 10
x = "hello"
x = [1, 2, 3]
```

The name `x` can be rebound to objects of different types. The *object* has a type; the *name* does not carry a fixed declared type.

Benefit

- ▶ fewer lines of codes
- ▶ generic functions over many object types

Operations are resolved at runtime

`a + b` asks: “what does addition mean for these objects?”

This enables [duck typing](#): If it walks like a duck and quacks like a duck, then it must be a duck.

Cost

- ▶ runtime (as opposed to compile-time) type errors
- ▶ performance overhead

Modern Python often adds type hints: `def f(x: int) -> int: ...`. These guide humans and tools; they are not the ordinary runtime execution model.

Programming Styles

Note: Object-based semantic model does not imply object-oriented programming style. Python supports multiple programming paradigms.

Procedural programming

=Imperative programming

Sequence of statements that change *program state*.

e.g.:

```
x = 0
x += 1
print(x)
```

Functional programming

Computation as evaluation of mathematical functions, avoiding mutable state.

e.g.:

```
def f(x):
    return x + 1
print(f(0))
```

OOP

Code organized around objects that bundle state and behavior.

e.g.

```
class Counter:
    def __init__(self):
        self.count = 0
    def increment(self):
        self.count += 1
c = Counter()
c.increment()
print(c.count) ⇒ 1
```

Modules and Packages

Levels of organization

- ▶ **Function:** reusable computation
- ▶ **Class:** reusable object type
- ▶ **Module:** one Python file
- ▶ **Package:** directory of modules
- ▶ **Project:** code, data, results, documentation

Project skeleton

```
my_project/  
  scripts/  
  src/  
    my_project/  
      __init__.py  
      data.py  
      models.py  
  results/  
  README.md
```

Import: `import numpy as np` searches for the package, initializes it if needed, and binds the name `np` to the module object. Then `np.array` is just attribute lookup on that object.

go back to google slides

Ways to use python

Vanilla:

- ▶ `python` command line
- ▶ `python script.py` script execution
- ▶ `python -i script.py` interactive mode script execution

Interactive:

- ▶ `ipython` command line (with tab completion, magic commands, etc.)
- ▶ `jupyter notebook` web-based interactive environment

IDEs:

- ▶ PyCharm, VSCode, Spyder, etc.

Scripts, Notebooks, and Packages

Script

`python train.py`

- ▶ executed top to bottom
- ▶ good for reproducible pipelines
- ▶ easy to rerun on cluster or terminal

Notebook

`analysis.ipynb`

- ▶ cells, plots, explanations
- ▶ good for exploration and teaching
- ▶ danger: hidden state and out-of-order execution

Package

`import my_project`

- ▶ reusable codebase
- ▶ clearer abstraction boundaries
- ▶ better tests and maintenance

Notebook for exploration. Script for execution. Package for reuse.

A good research workflow usually uses all three: explore in notebooks, stabilize logic into modules, run experiments through scripts.

The `__main__` Pattern

A good script separates definitions from execution

```
def load_data():  
    ...  
  
def train_model(data):  
    ...  
  
def main():  
    data = load_data()  
    train_model(data)  
  
if __name__ == "__main__":  
    main()
```

Why it matters

When a file is run directly, Python sets `__name__` to `"__main__"`.

When a file is imported, Python sets `__name__` to the module name.

Therefore this pattern prevents expensive experiment code from running accidentally during import.

Full Mental Model

- ▶ A Python program is a sequence of statements executed by an interpreter.
- ▶ Names point to objects; objects have identity, type, and value.
- ▶ Assignment binds names; mutation changes objects.
- ▶ Functions package computation; classes define new object types.
- ▶ Modules are files; packages are directories of modules; imports bind module objects to names.
- ▶ The standard library provides built-in tools; third-party packages extend the ecosystem.
- ▶ Virtual environments isolate dependencies; IDEs are workbenches around interpreters.

Place among languages

Systems languages

C, C++, Rust

- ▶ explicit control over memory/layout
- ▶ close to hardware and OS
- ▶ used for kernels, drivers, fast libraries

VM languages

Java, C#, JavaScript

- ▶ runtime manages memory
- ▶ strong tooling and deployment
- ▶ often bytecode or JIT execution

Scripting languages

Python, R, MATLAB

- ▶ productive, expressive, interactive
- ▶ strong data and plotting ecosystems
- ▶ often call compiled kernels

Shell / glue

Bash, PowerShell

- ▶ coordinate files and processes
- ▶ automate workflows
- ▶ command-line interaction

Research Python: Python is often the **control plane**. You write Python to express the experiment, analysis, model, or pipeline; the heavy numerical work may be executed by optimized low-level libraries underneath.

Python code → library call → C / Fortran /
CUDA kernel → result returned as Python object